

QUESTIONS

Q1

Find the bugs, include the missing lines in the following snippet.

1 mark



Q2

2. Buggy Code in an array

1 mark



Q3

3. Identify an errors in the following snippet

1 mark



3/3 answered

Q1 Find the bugs, include the missing lines in the following snippet.

```
int linearSearch(int arr[], int n, int key) {  
    for(int i = 0; i <= n; i++) {  
        if(arr[i] == key)  
            return i;  
    }  
    return -1;  
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the box below.

Your Fixed Code

```
1 int linearSearch(int arr[],int n,int key) {  
2     for (int i=0;i<n;i++) {  
3         if(arr[i]==key)  
4             return i;  
5     }  
6     return -1;  
7 }
```

QUESTIONS

Q1

Find the bugs, include the missing lines in the following snippet.

1 mark



Q2

2. Buggy Code in an array

1 mark



Q3

3. Identify an errors in the following snippet

1 mark



3/3 answered

Q2 2. Buggy Code in an array

```
#include <stdio.h>

int main() {
    int arr[5] = {10,20,30,40,50};
    for(int i=0;i<=5;i++)
    printf("%d ",arr[i]);
    return 0;
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code

Your Fixed Code

```
1 #include <stdio.h>
2 int main() {
3     int arr[5]={10,20,30,40,50};
4     for(int i=0;i<5;i++) {
5         printf("%d",arr[i]);
6     }
7     return 0;
8 }
```

Q1

Find the bugs, include the missing lines in the following snippet.

1 mark

**Q2**

2. Buggy Code in an array

1 mark

**Q3**

3. Identify an errors in the following snippet

1 mark



3/3 answered

```
int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n - 1;
    while(low < high)
    { int mid = (low + high) / 2;
      if(arr[mid] == key) return mid;
      else if(arr[mid] < key)
        low = mid;
      else high = mid;
    }
    return -1;
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the box below.

Your Fixed Code

```
1 int binarySearch(int arr[], int n, int key)
2 {
3     int low = 0, high = n - 1;
4     while (low <= high)
5     {
6         int mid = low + (high - low) / 2;
7         if (arr[mid] == key)
8             return mid;
9         else if (arr[mid] < key)
10            low = mid + 1;
11        else
12            high = mid - 1;
13    }
14    return -1;
15 }
```